TH4 — Decoder, Datapath, Manual Execution

CS2300 · Group G06
Chaithanya PS (CS25B018)
Yash Tulshyan (CS25B095)

This report covers `th4.circ`. The ALU (TH2) and the storage blocks (TH3) are merged into this file. TH4 adds `tara_decoder`, `IMM` and `tara_datapath`, which ties the program counter, MAR, MDR, IR, register file, ALU, accumulator, status register and `ram_word` together on one shared bus. Four manual runs were carried out on the datapath: an immediate load, a register-register arithmetic operation, a memory address calculation, and an instruction-fetch sequence over four memory words. Their screenshots are in Section 3, the compiled trace tables are in Section 3.6, and the control-signal inventory is in Section 4.

1. Interface table

Every new or modified subcircuit in `th4.circ` and its ports. Blocks carried over unchanged from TH1–TH3 (`register16`, `status_reg`, `ram_word`, `MAR`, `MDR`, `IR`, `addsub16` and so on) keep the interfaces given in the earlier reports.

Subcircuit	Port	Width	Notes
tara_decoder	IR (in)	16	Instruction word from the IR register. Purely combinational
	OPCODE	5	IR[15:11]
	FORMAT	3	Format number 0–7, chosen from OPCODE by a 32-input multiplexer of constants
	RD	3	IR[10:8]
	RA	3	IR[7:5]
	RB	3	IR[4:2]
	IMMA8	8	IR[7:0] (the pin is labelled `IMMA8` in the circuit)
	REL8	8	IR[7:0]
	OFF5	5	IR[4:0]
	REL11	11	IR[10:0]
IMM	IMM8	8	Immediate byte
	IMM5	5	5-bit offset / immediate
	BIT_EXTEND	2	00 zero-extend IMM8; 01 sign-extend IMM8; 10 IMM8 into bits 15:8 with low byte 0; 11 zero-extend IMM5. (Pin is `IMM_BIT_EXTEND` on the datapath)
	IMM_OUT	16	Extended value; bus source 011
tara_datapath inputs	BUS_SOURCE	3	Encoded bus-source select (Section 2.2)
	CLK	1	Common clock (also clocks `ram_word`)
	MAR_LOAD, MDR_LOAD, IR_LOAD, ACC_LOAD	1 each	Load enables of MAR, MDR, IR and ACC
	PC_LOAD, PC_FETCH_EN, PC_RST	1 each	Program-counter controls (Section 2.6)

Subcircuit	Port	Width	Notes
	MEM_READ, MEM_WRITE	1 each	Read and write enables of the internal `ram_word`
	REG_A, REG_B, REG_D	3 each	Read-port A index, read-port B index, write index
	REG_WE, REG_RST	1 each	Register-file write enable and reset
	IMM8, IMM5, IMM_BIT_EXTEND	8, 5, 2	Inputs of IMM
	OPCODE	5	ALU operation code
	STATUS_WE	1	Write enable of the status register (Z, N, C, V)
tara_datapath outputs	BUS_VALUE	16	The shared bus itself
	R0 – R7	16 each	Register-file contents (probes for the trace sheet)
	PC_OUT, MAR_OUT, MDR_OUT, IR_OUT, IMM_OUT, ACC_OUT	16 each	Register and block outputs
	RAM_WORD_OUT	16	`ram_word` DOUT
	ALU_OUT	16	ALU result
	Z_OUT, N_OUT, C_OUT, V_OUT	1 each	Status-register outputs
regfile8x16 (modified)	RA, RB, RD	3 each	Read A, read B and write indices
	D, WE, CLK, RST	16, 1, 1, 1	Write data comes straight from the shared bus; there is no EXT input and no multiplexer in front of D
	QA, QB, R0 – R7	16 each	Read ports and per-register outputs
PC (modified)	FETCH, CLK, PC_LOAD, FETCH_EN, RST	16, 1, 1, 1, 1	Gains the PC_LOAD control (Section 2.6)
	P_C	16	Program-counter value
alu (modified)	OPCODE, A, B, ALU_OUTPUT_ENABLE	5, 16, 16, 1	Outputs are gated by ALU_OUTPUT_ENABLE; the datapath ties it to constant 1
	Z, N, C, V, BUS_OUT	1, 1, 1, 1, 16	Condition flags and result

2. Design decisions

2.1 Decoder

`tara_decoder` is combinational. A single 16-bit splitter exposes every physical field at once, so the fields overlap: OPCODE = IR[15:11], RD = IR[10:8], RA = IR[7:5], RB = IR[4:2], OFF5 = IR[4:0], IMM8 = REL8 = IR[7:0] and REL11 = IR[10:0]. Which of them is meaningful depends on the instruction format, so FORMAT is produced separately: a 32-input multiplexer addressed by OPCODE, whose inputs are constants in the range 0–7. The control unit in TH5 will use FORMAT to decide which fields to read.

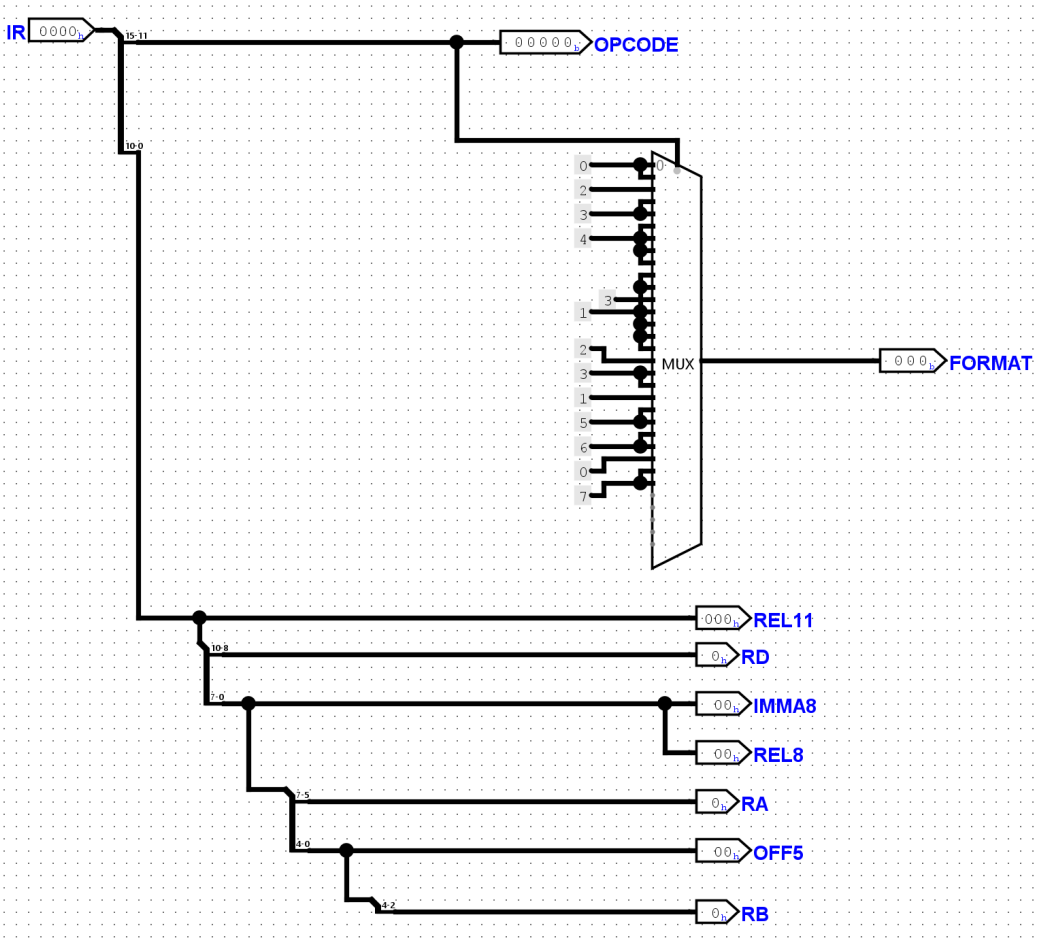


Figure 1 — tara_decoder

2.2 Shared bus and bus-source select

Seven sources share the 16-bit bus, each behind a controlled (tri-state) buffer. Rather than seven independent enables, the datapath uses an encoded 3-bit `BUS_SOURCE` that goes through a 3-to-8 decoder whose enable is tied high. Only one decoder output can be high at a time, so two sources driving the bus simultaneously is structurally impossible. Code 111 selects no source, which turns the bus off.

BUS_SOURCE	Source	Probe
000	PC	PC_OUT
001	MDR	MDR_OUT
010	IR	IR_OUT
011	IMM (extended immediate or offset)	IMM_OUT
100	Register file, read port A	R0–R7 / QA
101	ACC	ACC_OUT
110	ram_word (RamWord read)	RAM_WORD_OUT
111	Bus off (no source)	—

Destinations take their value from the bus on the rising clock edge when their load enable is set: MAR (`MAR_LOAD`), MDR (`MDR_LOAD`), the register-file write port (`REG_WE`) and the PC (`PC_LOAD`). All four are exercised in Section 3.

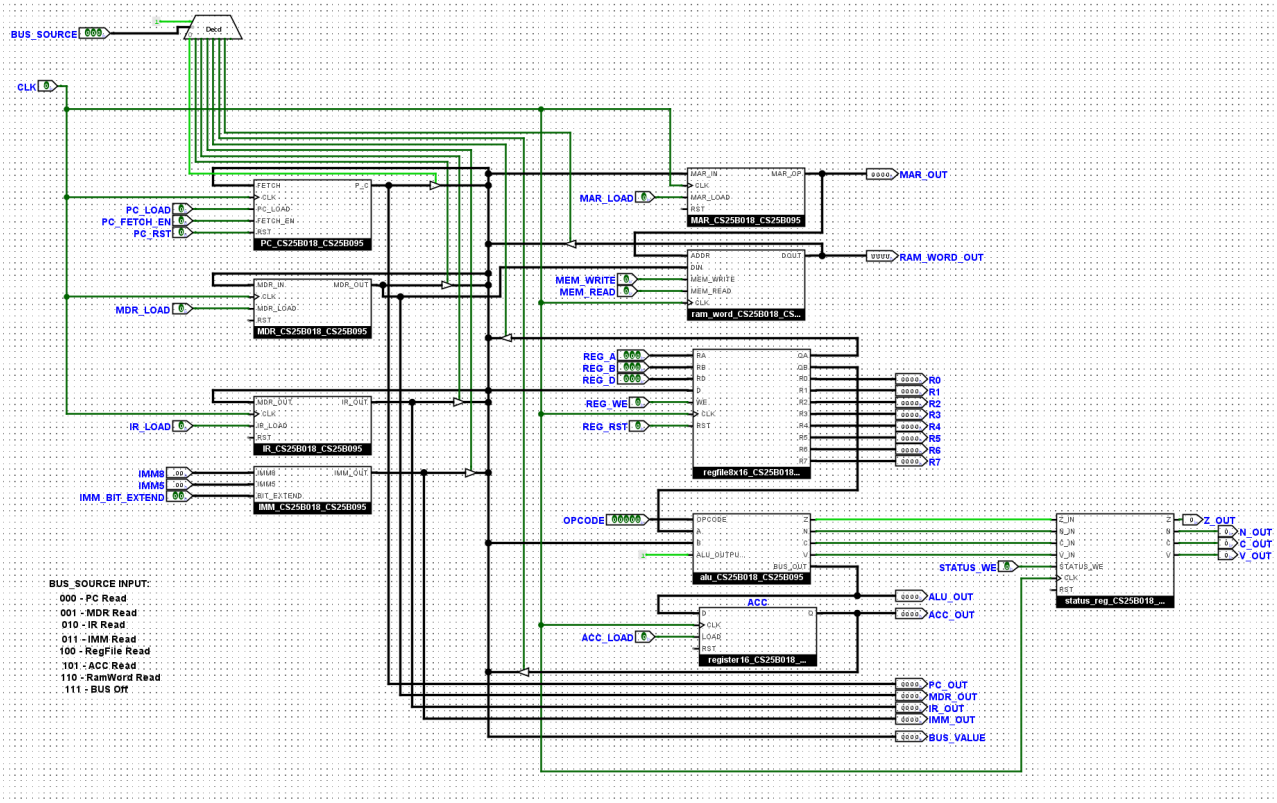


Figure 2 — tara_datapath (inside)

2.3 ALU operand routing, accumulator and status register

The ALU's A input is the shared bus, and its B input is register-file read port B. The ALU result is **not** placed on the shared bus. It is captured in ACC, an instance of `register16`, when `ACC_LOAD` = 1, and ACC can then be driven back onto the bus as source 101. Operand and result therefore never compete for the bus in the same state: in Manual execution 2, BUS_VALUE = 0x36 (operand A) while ALU_OUT = 0x74. A register-register operation takes two manual states, execute (ACC ← A op B) and write-back (bus ← ACC, RD ← bus).

The ALU condition outputs Z, N, C and V feed a `status_reg`, which captures them on the clock edge when `STATUS_WE` = 1 and drives Z_OUT, N_OUT, C_OUT and V_OUT. The ALU's output-enable pin is tied to constant 1 inside the datapath, so the ALU is always enabled. `STATUS_WE` is held at 0 in the manual runs of Section 3, so the flag outputs stay at 0 throughout.

2.4 Memory inside the datapath

`ram_word` (TH3) is instantiated inside `tara_datapath`, so fetch and memory states can be run from the panel with no external wiring. Its ADDR is MAR_OUT, its DIN is MDR_OUT, and its DOUT goes to the RAM_WORD_OUT probe and to bus source 110. It is clocked by the datapath CLK. `MEM_READ` and `MEM_WRITE` are 1-bit panel switches.

- **Read:** MAR must already hold the address. `MEM_READ` = 1 makes the addressed word appear on RAM_WORD_OUT; with `BUS_SOURCE` = 110 it is on the bus and `MDR_LOAD` = 1 captures it into MDR (Section 3.5).
- **Before any read:** with `MEM_READ` = 0 and no word read yet, RAM_WORD_OUT reads U (Figure 4).
- **Write:** `MEM_WRITE` = 1 stores MDR_OUT at address MAR_OUT on the clock edge. `MEM_READ` and `MEM_WRITE` should never be asserted together.
- **Addressing:** byte-addressed and little-endian as decided in TH3; ADDR[0] is dropped inside `ram_word`, so successive words sit at addresses 0, 2, 4, 6 (Section 3.5).

2.5 Immediate and offset block

`IMM` produces the 16-bit value for bus source 011. A 2-bit `BIT_EXTEND` chooses between four modes: zero-extension of IMM8 (00), sign-extension of IMM8 (01), IMM8 placed in the high byte with a zero low byte (10), and zero-extension of the separate 5-bit IMM5 input (11). **Decision:** the 5-bit offset is zero-extended, so an effective address is Rb + (0 to 31). This is recorded here so that TH6 uses the same convention.

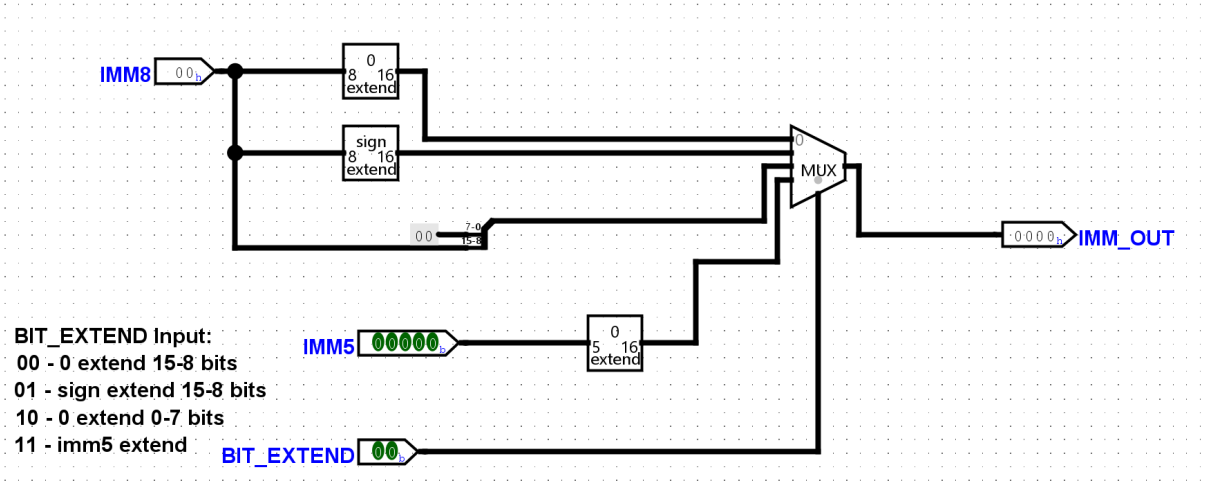


Figure 3 — IMM subcircuit

2.6 Program counter

The PC now has a `PC_LOAD` control. With `PC_LOAD` = 0 it holds its value: PC_OUT stays at 0x0000 through all clock pulses of the three manual executions in Sections 3.2–3.4. With `PC_LOAD` = 1 the PC advances by 2 on the clock edge, as seen in the fetch run (PC_OUT 0 → 2 → 4 → 6 → 8, Section 3.5). `PC_FETCH_EN` is held at 0 and `PC_RST` at 0 in every run, so the load-from-bus path is not exercised in TH4.

2.7 Changes made to earlier blocks

- **regfile8x16 (TH3):** the EXT input and the multiplexer in front of D are not present. The write data is simply the bus, and a move is “source register onto the bus, destination register writes the bus”.
- **alu (TH2):** the outputs are gated by `ALU_OUTPUT_ENABLE`, which the datapath ties to 1. A is the shared bus and B is register-file port B.
- **PC (TH3):** `PC_LOAD` added (Section 2.6).
- **ACC (new):** a `register16` instance with `ACC_LOAD`, added as the ALU result register (Section 2.3). The Z, N, C, V outputs go to a `status_reg` instance written with `STATUS_WE`.

3. Test evidence

In every screenshot the magenta ring marks the input that was changed for that step. Probe values are shown in hexadecimal. A clock pulse is a 0→1 transition of CLK. Unless listed, all other panel inputs are at their inactive value (Section 4).

3.1 Initial state

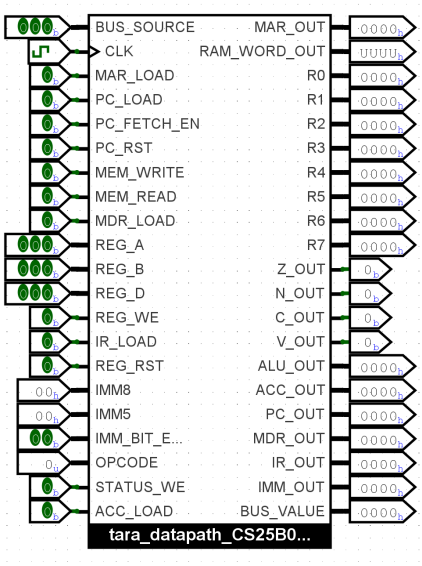


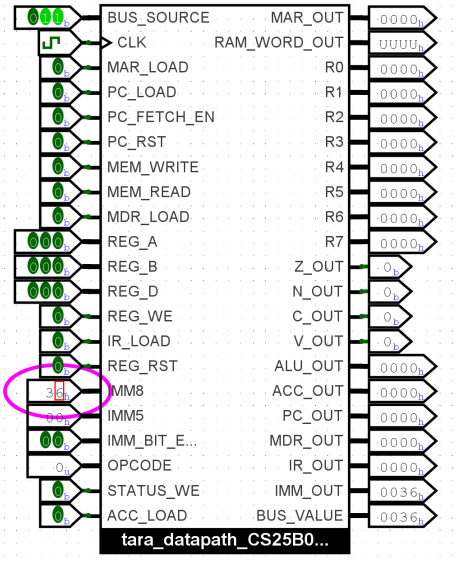
Figure 4 — tara_datapath, initial state

Stimulus: every panel input at its inactive value (all loads, resets, `MEM_READ`, `MEM_WRITE` and `STATUS_WE` 0, all indices 000, `BUS_SOURCE` = 000), clock low.

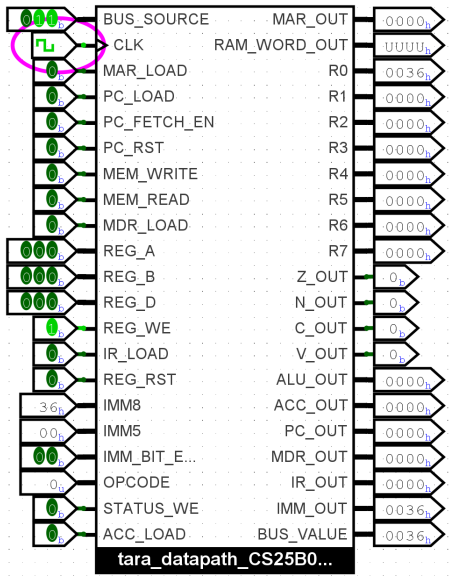
Observed: R0–R7, PC_OUT, MAR_OUT, MDR_OUT, IR_OUT, ACC_OUT, ALU_OUT, IMM_OUT and Z/N/C/V all read 0. BUS_VALUE reads 0x0000 because `BUS_SOURCE` = 000 selects the PC. RAM_WORD_OUT reads **U** because no read has been issued yet.

3.2 Manual execution 1: immediate load, LIL R0, 0x36

#	Stimulus	Expected / observed
1	`BUS_SOURCE`=011 (IMM), `IMM8`=0x36, `IMM_BIT_EXTEND`=00, `REG_D`=000; clock low	IMM_OUT = BUS_VALUE = 0x0036; R0 still 0
2	`REG_WE`=1; clock 0→1	R0 = 0x0036 (54); PC_OUT stays 0



Step 1 — IMM on the bus, IMM8 = 0x36 (ringed)

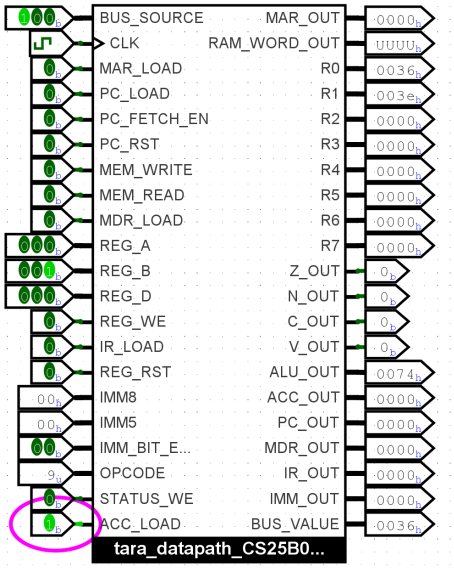


Step 2 — REG_WE = 1, clock pulse: R0 = 0x0036

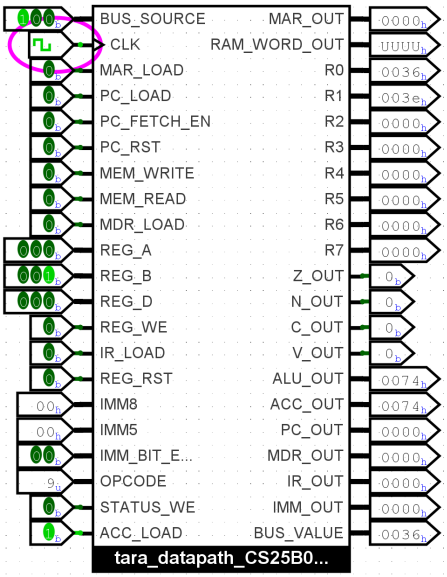
3.3 Manual execution 2: register-register arithmetic, ADD R3, R0, R1

R0 = 0x36 from Manual execution 1. R1 = 0x3E (62) had been loaded in the same way (`IMM8` = 0x3E, `REG_D` = 001); that set-up step is not shown separately, but R1 reads 0x003E in the first screenshot below.

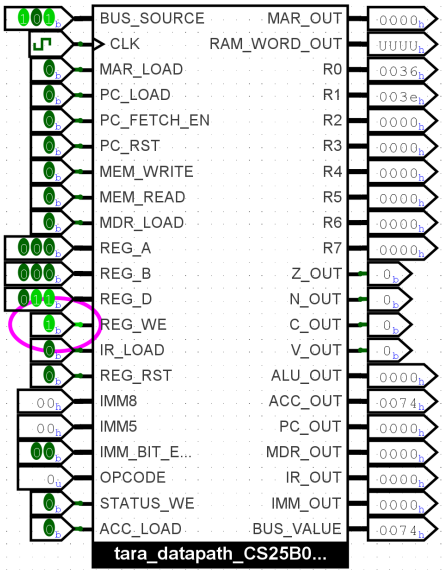
#	Stimulus	Expected / observed
1	`BUS_SOURCE`=100 (register file port A), `REG_A`=000, `REG_B`=001, `OPCODE`=9 (ADD), `ACC_LOAD`=1	BUS_VALUE = 0x0036 (R0 as operand A); ALU_OUT = 0x0074; Z=N=C=V=0; ACC_OUT still 0
2	Clock 0→1	ACC_OUT = 0x0074 (116)
3	`BUS_SOURCE`=101 (ACC), `ACC_LOAD`=0, `OPCODE`=0, `REG_D`=011, `REG_WE`=1	BUS_VALUE = 0x0074; R3 still 0
4	Clock 0→1	R3 = 0x0074 (116); ACC_OUT still 0x0074



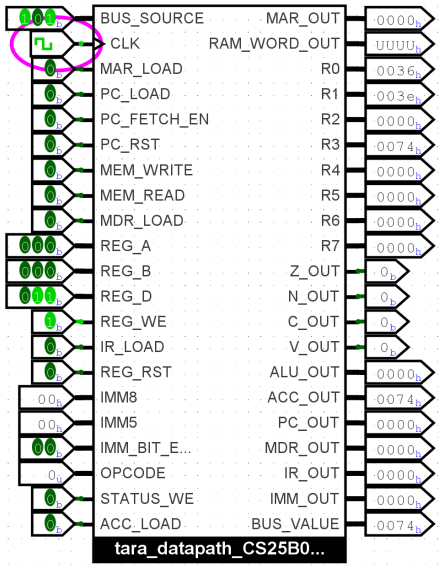
Step 1 — ADD set up, ACC_LOAD raised (ringed)



Step 2 — clock pulse: ACC_OUT = 0x0074



Step 3 — ACC on the bus, REG_D = 011, REG_WE raised

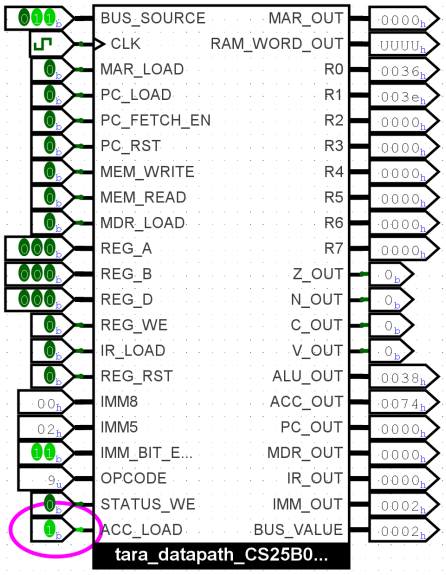


Step 4 — clock pulse: R3 = 0x0074

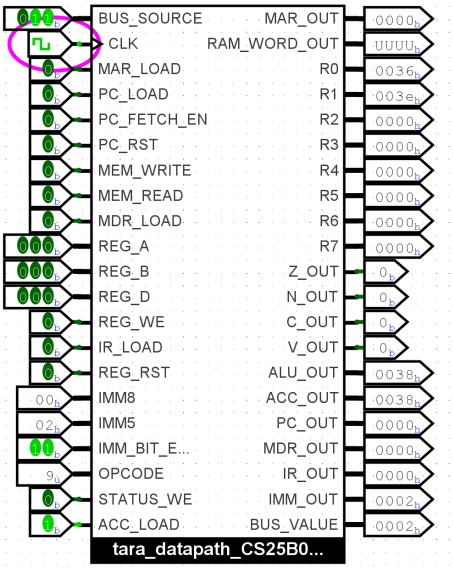
3.4 Manual execution 3: memory address calculation, EA = R0 + 2

This is the effective-address phase of a load such as LDW Rd, 2(R0): the offset comes from IMM (IMM5 path), the base from R0 on port B, and the sum is latched in ACC and then loaded into MAR. R0 = 0x36 as before.

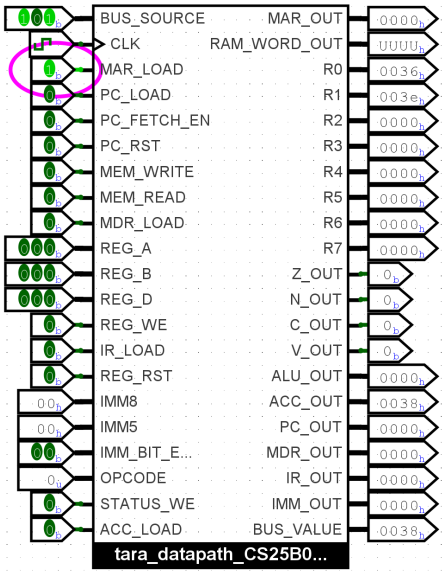
#	Stimulus	Expected / observed
1	BUS_SOURCE=011 (IMM), IMM5=2, IMM_BIT_EXTEND=11, REG_B=000, OPCODE=9 (ADD), ACC_LOAD=1	IMM_OUT = BUS_VALUE = 0x0002; ALU_OUT = 0x0038; Z=N=C=V=0; ACC_OUT still 0x0074 from Manual execution 2
2	Clock 0→1	ACC_OUT = 0x0038 (56)
3	BUS_SOURCE=101 (ACC), IMM5=0, IMM_BIT_EXTEND=00, OPCODE=0, ACC_LOAD=0, MAR_LOAD=1	BUS_VALUE = 0x0038; MAR_OUT still 0
4	Clock 0→1	MAR_OUT = 0x0038; ACC_OUT still 0x0038



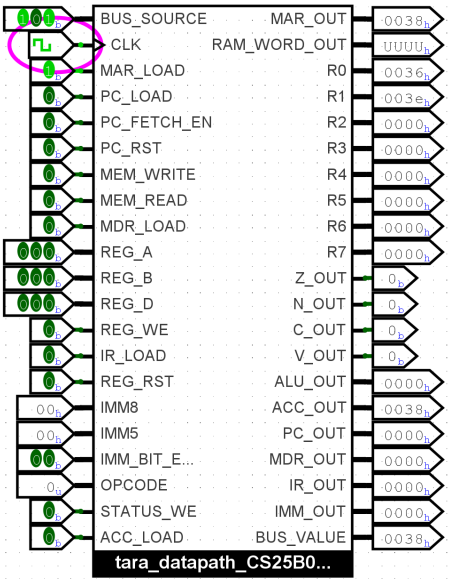
Step 1 — offset on the bus, ADD with R0 (ACC_LOAD ringed)



Step 2 — clock pulse: ACC_OUT = 0x0038



Step 3 — ACC on the bus, MAR_LOAD raised

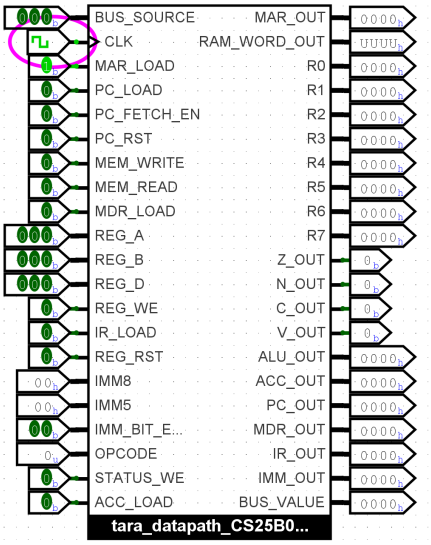


Step 4 — clock pulse: MAR_OUT = 0x0038

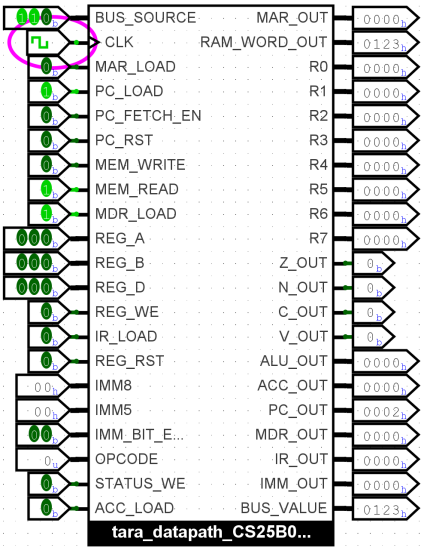
3.5 Manual execution 4: instruction fetch from ram_word

`ram_word` holds the test words 0x0123, 0x4567, 0x89AB and 0xCDEF at addresses 0, 2, 4 and 6. Each fetch has the same actions: **MAR ← PC** (`BUS_SOURCE`=000, `MAR_LOAD`=1); **read** (`MEM_READ`=1, word appears on RAM_WORD_OUT); **MDR ← memory and PC ← PC+2** (`BUS_SOURCE`=110, `MDR_LOAD`=1, `PC_LOAD`=1); **IR ← MDR** (`IR_LOAD`=1). In fetch 1 the read, PC advance and MDR load were set together: the PC advanced at that pulse and MDR took the word one pulse later, hence the extra step. Fetches 2–4 follow the pattern above; fetch 4 is shown again to confirm it repeats. The full sequence is in Section 3.6.

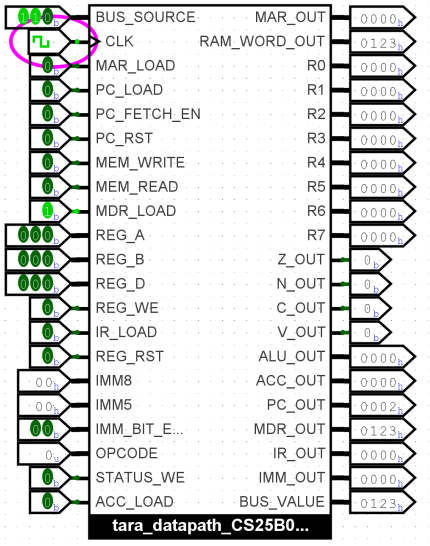
Fetch 1 (address 0)



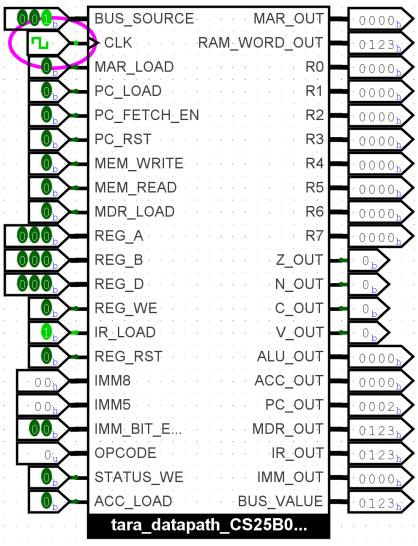
F1.1 — MAR_LOAD with PC on the bus: MAR_OUT = 0x0000



F1.2 — memory read, PC_LOAD, MDR_LOAD set: RAM_WORD_OUT = 0x0123, PC_OUT = 0x0002

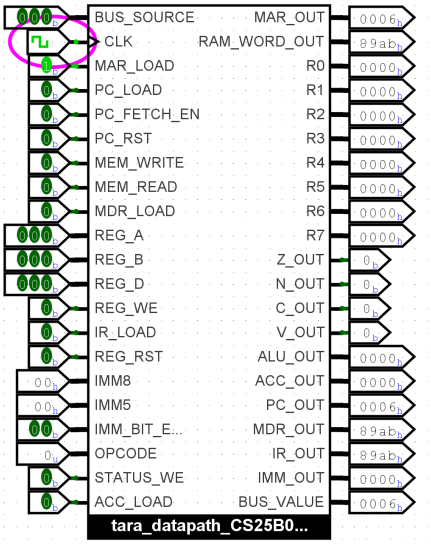


F1.3 — MDR_LOAD with RamWord on the bus: MDR_OUT = 0x0123

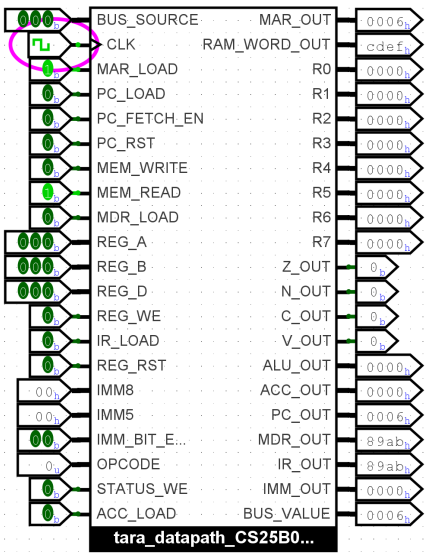


F1.4 — IR_LOAD with MDR on the bus: IR_OUT = 0x0123

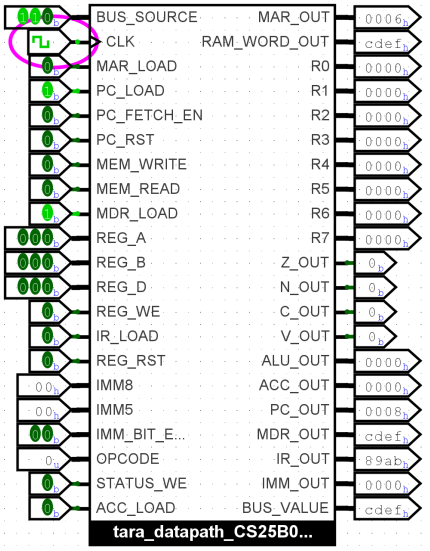
Fetch 4 (address 6)



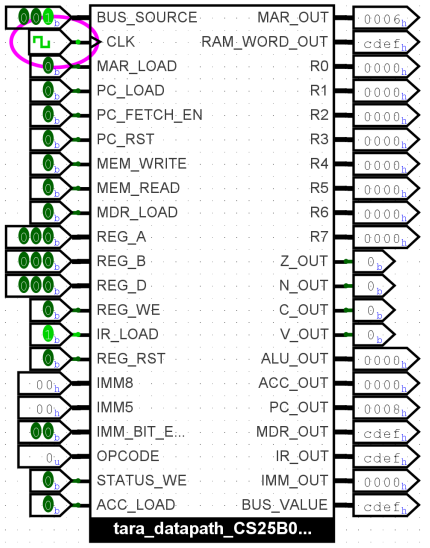
F4.1 — MAR_LOAD with PC on the bus: MAR_OUT = 0x0006



F4.2 — MEM_READ: RAM_WORD_OUT = 0xCDEF



F4.3 — MDR_LOAD, PC_LOAD: MDR_OUT = 0xCDEF, PC_OUT = 0x0008



F4.4 — IR_LOAD: IR_OUT = 0xCDEF

3.6 Trace tables

Compiled from the screenshots above. Only signals that differ from their inactive value are listed. The clock is pulsed once per step.

Step	State / micro-operation	Panel signals set	After the clock edge
Manual execution 1 — LIL R0, 0x36 (R0 ← 0x0036)			
1	R0 ← IMM8	`BUS_SOURCE`=011, `IMM8`=0x36, `IMM_BIT_EXTEND`=00, `REG_D`=000, `REG_WE`=1	R0 = 0x0036; PC_OUT 0
Manual execution 2 — ADD R3, R0, R1 (R3 ← R0 + R1)			
2	Execute: ACC ← R0 + R1	`BUS_SOURCE`=100, `REG_A`=000, `REG_B`=001, `OPCODE`=9, `ACC_LOAD`=1	ACC = 0x0074; Z=N=C=V=0
3	Write-back: R3 ← ACC	`BUS_SOURCE`=101, `REG_D`=011, `REG_WE`=1	R3 = 0x0074
Manual execution 3 — memory address calculation (MAR ← R0 + 2)			

Step	State / micro-operation	Panel signals set	After the clock edge
4	Execute: ACC ← IMM5 + R0	BUS_SOURCE=011, IMM5=2, IMM_BIT_EXTEND=11, REG_B=000, OPCODE=9, ACC_LOAD=1	ACC = 0x0038
5	MAR ← ACC	BUS_SOURCE=101, MAR_LOAD=1	MAR = 0x0038
Manual execution 4 — four fetches from ram_word			
F1.1	MAR ← PC	BUS_SOURCE=000, MAR_LOAD=1	MAR = 0x0000
F1.2	Read word at 0; PC ← PC+2	BUS_SOURCE=110, MEM_READ=1, PC_LOAD=1, MDR_LOAD=1	RAM_WORD_OUT = 0x0123; PC = 0x0002; MDR = 0x0000
F1.3	MDR ← memory	BUS_SOURCE=110, MDR_LOAD=1	MDR = 0x0123
F1.4	IR ← MDR	BUS_SOURCE=001, IR_LOAD=1	IR = 0x0123
F2.1	MAR ← PC	BUS_SOURCE=000, MAR_LOAD=1	MAR = 0x0002
F2.2	Read word at 2	BUS_SOURCE=000, MAR_LOAD=1, MEM_READ=1	RAM_WORD_OUT = 0x4567
F2.3	MDR ← memory; PC ← PC+2	BUS_SOURCE=110, MDR_LOAD=1, PC_LOAD=1	MDR = 0x4567; PC = 0x0004
F2.4	IR ← MDR	BUS_SOURCE=001, IR_LOAD=1	IR = 0x4567
F3.1	MAR ← PC	BUS_SOURCE=000, MAR_LOAD=1	MAR = 0x0004
F3.2	Read word at 4	BUS_SOURCE=000, MAR_LOAD=1, MEM_READ=1	RAM_WORD_OUT = 0x89AB
F3.3	MDR ← memory; PC ← PC+2	BUS_SOURCE=110, MDR_LOAD=1, PC_LOAD=1	MDR = 0x89AB; PC = 0x0006
F3.4	IR ← MDR	BUS_SOURCE=001, IR_LOAD=1	IR = 0x89AB
F4.1	MAR ← PC	BUS_SOURCE=000, MAR_LOAD=1	MAR = 0x0006
F4.2	Read word at 6	BUS_SOURCE=000, MAR_LOAD=1, MEM_READ=1	RAM_WORD_OUT = 0xCDEF
F4.3	MDR ← memory; PC ← PC+2	BUS_SOURCE=110, MDR_LOAD=1, PC_LOAD=1	MDR = 0xCDEF; PC = 0x0008
F4.4	IR ← MDR	BUS_SOURCE=001, IR_LOAD=1	IR = 0xCDEF

4. Control-signal inventory

Every input on the manual panel, the component it enables or loads, and its inactive value. This is the specification of the control word for TH5. With every load, write and reset input at 0, no register changes state.

Signal	Width	Drives	Function	Inactive
BUS_SOURCE	3	Bus-source decoder	Selects the bus driver: 000 PC, 001 MDR, 010 IR, 011 IMM, 100 regfile port A, 101 ACC, 110 ram_word, 111 none	000 (111 = bus off)
CLK	1	PC, MAR, MDR, IR, regfile, ACC, status_reg, ram_word	Common clock; all captures occur on the rising edge	0
MAR_LOAD	1	MAR	Load MAR from the bus	0
PC_LOAD	1	PC	1: PC advances by 2 on the clock edge. 0: PC holds	0
PC_FETCH_EN	1	PC	Selects the PC's FETCH input path (not exercised in TH4)	0
PC_RST	1	PC	Clear PC	0
MEM_WRITE	1	ram_word	Store MDR_OUT at address MAR_OUT on the clock edge. Never together with MEM_READ	0

Signal	Width	Drives	Function	Inactive
MEM_READ	1	ram_word	Read the addressed word onto RAM_WORD_OUT / bus source 110	0
MDR_LOAD	1	MDR	Load MDR from the bus	0
IR_LOAD	1	IR	Load IR from MDR	0
REG_A	3	regfile port A	Index of the register read on port A (drives the bus when BUS_SOURCE = 100)	000
REG_B	3	regfile port B	Index of the register read on port B (feeds ALU input B)	000
REG_D	3	regfile write port	Index of the destination register	000
REG_WE	1	regfile	Write the bus value into register REG_D on the clock edge	0
REG_RST	1	regfile	Clear R0–R7	0
IMM8	8	IMM	Immediate byte	0x00
IMM5	5	IMM	5-bit offset (zero-extended in mode 11)	0x00
IMM_BIT_EXTEND	2	IMM	00 zero-extend IMM8, 01 sign-extend IMM8, 10 byte into bits 15:8, 11 zero-extend IMM5	00
OPCODE	5	alu	ALU operation code (for example 9 = ADD)	0
ACC_LOAD	1	ACC	Capture the ALU result into ACC	0
STATUS_WE	1	status_reg	Capture Z, N, C, V from the ALU	0

5. Faults encountered

Fault 1 — ACC had no reset input

The first version of the datapath gave `MAR`, `PC`, `MDR`, `IR` and the register file a reset input each, but the accumulator had none. Once a result had been latched into `ACC` there was no way to clear it. `ACC_RST` was added to the `ACC` register and brought out to the manual panel, and it now appears in the control-signal inventory.

Fault 2 — the 5-bit offset was being fed through IMM8

The memory address calculation needs `Rb` + `off5`. The first attempt fed the offset into `IMM` through the `IMM8` pin. That does not work cleanly: `IMM8` is 8 bits wide and its extension modes are chosen by `IMM_BIT_EXTEND`, none of which handles a 5-bit value. The extension would have had to be done outside the block, either by hand on the control panel or with extra logic in `tara_datapath`, which duplicates the job `IMM` exists to do. The fix was a dedicated 5-bit `OFFSET` input on `IMM` (zero-extended) with a select, `I8_OR_OFF5`. Test 3 now shows the offset arriving on the bus as 2 (`IMM_OUT` = 2) with no extra steps.

Fault 3 — the EXT multiplexer on the register-file write data

In TH3 `regfile8x16` had an `EXT` input choosing between external write data and the value read on port A, so that `MOV` could complete in one clock. With a shared bus this multiplexer is redundant: a move is the source register on the bus and the destination register writing the bus. Keeping `EXT` added a panel switch and made `MOV` need a different setting from every other write, which was more complicated than intended. `EXT` and the multiplexer were removed and `D` is wired to the bus. The write-back in Test 2 (ACC → bus → R3) and the load in Test 1 (IMM → bus → R0) now use exactly the same path.